

Self Attention QKV

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes

Self-Attention in Transformers: Query, Key, and Value (Q, K, V)

১. 🧠 সহজ ভাষায় পরিচিতি (Intuition)

এই concept কী এবং কেন দরকার?

কল্পনা করো তুমি একটি বাংলা বাক্য পড়ছো:

"আমি গতকাল বাজারে গিয়েছিলাম এবং সেখানে অনেক তাজা মাছ কিনলাম।"

এই বাক্যে "সেখানে" শব্দটি বুঝতে হলে তোমাকে পিছনে গিয়ে "বাজারে" শব্দটি মনে করতে হবে। মানুষের মস্তিষ্ক এটি স্বাভাবিকভাবেই করে — বাক্যের কোন শব্দটি কোন শব্দের সাথে সম্পর্কিত তা বুঝতে পারে।

Self-Attention হলো Transformer-এর সেই ক্ষমতা যেটি একটি বাক্যের প্রতিটি শব্দকে বাকি সব শব্দের সাথে তুলনা করে বলে দেয় — "এই শব্দটি বোঝার জন্য অন্য কোন কোন শব্দগুলো সবচেয়ে বেশি গুরুত্বপূর্ণ?"

🛒 বাস্তব জীবনের উদাহরণ: গ্রন্থাগারের বই খোঁজা!

মনে করো তুমি একটি বড় **পাবলিক লাইব্রেরিতে** গিয়েছো। তোমার "রবীন্দ্রনাথের কবিতা" বিষয়ক একটি বই দরকার।

তুমি যা করো (Query):

- তুমি লাইব্রেরিয়ানকে বলো: "আমার রবীন্দ্রনাথের কবিতার বই দরকার।" — এটি হলো তোমার **Query (প্রশ্ন/অনুসন্ধান)**।

লাইব্রেরির প্রতিটি বই যা বহন করে (Key):

- প্রতিটি বইয়ের র‍্যাঙ্কে একটি **লেবেল/ট্যাগ** থাকে: "রবীন্দ্রনাথ", "গণিত", "ইতিহাস" ইত্যাদি। — এই ট্যাগগুলোই হলো **Key** (পরিচয়)।

বইয়ের ভেতরের আসল তথ্য (Value):

- তুমি ট্যাগ মিলিয়ে যে বই বের করো, সেই বইয়ের **আসল বিষয়বস্তু** হলো **Value** (মান/বিষয়বস্তু)।

Self-Attention মানে:

প্রতিটি শব্দ নিজেই অন্য সব শব্দের কাছে এই প্রশ্ন করে: _"তুমি আমার জন্য কতটা relevant (প্রাসঙ্গিক)?"_

এটি কোন সমস্যা সমাধান করে?

পুরনো RNN/LSTM-এর সমস্যা:

- শুধু একটি দিক থেকে (বাম থেকে ডান) বাক্য প্রক্রিয়া করত।
- দীর্ঘ বাক্যে দূরের শব্দগুলোর সম্পর্ক মনে রাখতে পারত না।
- Sequential (ধারাবাহিক) হওয়ায় Parallel processing সম্ভব ছিল না।

Self-Attention যা করে:

- একবারেই বাক্যের **সব শব্দ পরস্পরের সাথে** তুলনা করে।
- দূরত্ব নির্বিশেষে যেকোনো দুটি শব্দের সম্পর্ক ধরতে পারে।
- Parallel processing সম্ভব, তাই অনেক দ্রুত।

২. বিস্তারিত ব্যাখ্যা (Deep Explanation)

Self-Attention কীভাবে কাজ করে?

ধরো আমাদের বাক্য: **"The cat sat on the mat"**

এই বাক্যে ৬টি শব্দ আছে। Self-Attention এই প্রতিটি শব্দের জন্য জিজ্ঞেস করে:

"এই শব্দটি বোঝার জন্য বাক্যের অন্য কোন শব্দগুলো সবচেয়ে গুরুত্বপূর্ণ?"

ধাপ ১: Word Embedding থেকে শুরু

প্রতিটি শব্দ একটি সংখ্যার ভেক্টর (Embedding) হিসেবে represent হয়।

"cat" $\rightarrow [0.2, 0.8, 0.1, 0.5]$ (4-dimensional vector)

"sat" $\rightarrow [0.6, 0.3, 0.9, 0.2]$

"mat" $\rightarrow [0.1, 0.7, 0.4, 0.8]$

ধাপ ২: তিনটি Weight Matrix দিয়ে Q, K, V তৈরি

প্রতিটি Embedding Vector তিনটি আলাদা Learnable Weight Matrix দিয়ে গুণ করা হয়:

- **W_Q** (Query Weight Matrix)
- **W_K** (Key Weight Matrix)
- **W_V** (Value Weight Matrix)

Query = Embedding $\times$ W_Q $\rightarrow$ "আমি কী খুঁজছি?"

Key = Embedding $\times$ W_K $\rightarrow$ "আমার কাছে কী আছে?"

Value = Embedding $\times$ W_V $\rightarrow$ "আমার আসল content কী?"

ধাপ ৩: Attention Score গণনা (Dot Product)

"cat" শব্দের Query vector কে বাকি সব শব্দের Key vector-এর সাথে তুলনা করি:

Score("cat", "sat") = Query_cat $\cdot$ Key_sat (Dot Product)

Score("cat", "mat") = Query_cat $\cdot$ Key_mat

Score("cat", "the") = Query_cat $\cdot$ Key_the

...

Dot Product যত বেশি $\rightarrow$ দুটি শব্দের সম্পর্ক তত বেশি।

ধাপ ৪: Scaling (ভাগ করা)

Dot Product-এর মান অনেক বড় হতে পারে, যা Softmax-কে unstable করে। তাই $\sqrt{d_k}$ দিয়ে ভাগ করি:

Scaled Score = Score / $\sqrt{d_k}$

যেখানে d_k = Key vector-এর dimension।

ধাপ ৫: Softmax (Probability তে রূপান্তর)

Scaled Score গুলোকে Softmax দিয়ে 0 থেকে 1-এর মধ্যে নিয়ে আসি, এবং সব মিলে 1 হয়:

Attention Weights = Softmax(Scaled Scores)

উদাহরণ: [0.6, 0.1, 0.2, 0.05, 0.05] → যোগফল = 1.0

এই weights মানে: "cat" শব্দটি বোঝার জন্য প্রথম শব্দে 60%, তৃতীয় শব্দে 20% মনোযোগ দাও।

ধাপ ৬: Weighted Sum of Values (চূড়ান্ত আউটপুট)

Attention Weights দিয়ে Value vectors-গুলো যোগ করি:

Output_cat = 0.6 × Value_1 + 0.1 × Value_2 + 0.2 × Value_3 + ...

এই চূড়ান্ত ভেক্টরটি "cat" শব্দের **context-aware representation** — যেটি বাকি সব শব্দের তথ্য ধারণ করে।

৩. Math / Theory

মূল Self-Attention সূত্র

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

প্রতিটি Symbol-এর অর্থ:

Symbol	অর্থ	আকার
Q	Query Matrix — "কী খুঁজছি?"	$(n \times d_k)$
K	Key Matrix — "কে কী ধরনের?"	$(n \times d_k)$
V	Value Matrix — "কার কাছে কী আছে?"	$(n \times d_v)$
d_k	Key vector-এর Dimension	scalar
n	বাক্যে শব্দের সংখ্যা	scalar
QK^T	Query এবং Key-এর Dot Product	$(n \times n)$
$\text{Softmax}(\dots)$	Attention Weights (probability)	$(n \times n)$
Output	Context-aware Representation	$(n \times d_v)$

 সহজ সংখ্যা দিয়ে Manual Calculation

ধরো আমাদের ২টি শব্দ আছে: "আমি" এবং "খাই"

- $d_k = 2$ (ছোট উদাহরণের জন্য)

Step 1: Embedding vectors

আমি $\rightarrow x_1 = [1, 0]$

খাই $\rightarrow x_2 = [0, 1]$

Step 2: Weight Matrices (random initialized)

$W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ (Identity matrix, সহজের জন্য)

$W_K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

$W_V = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

Step 3: Q, K, V তৈরি

$$Q1 = x1 \times W_Q = [1, 0] \quad (\text{আমি-র Query})$$

$$Q2 = x2 \times W_Q = [0, 1] \quad (\text{খাই-এর Query})$$

$$K1 = x1 \times W_K = [1, 0] \quad (\text{আমি-র Key})$$

$$K2 = x2 \times W_K = [0, 1] \quad (\text{খাই-এর Key})$$

$$V1 = x1 \times W_V = [1, 2] \quad (\text{আমি-র Value})$$

$$V2 = x2 \times W_V = [3, 4] \quad (\text{খাই-এর Value})$$

Step 4: "আমি"-র জন্য Attention Score

$$\text{Score}(\text{আমি}, \text{আমি}) = Q1 \cdot K1 = [1, 0] \cdot [1, 0] = 1 \times 1 + 0 \times 0 = 1$$

$$\text{Score}(\text{আমি}, \text{খাই}) = Q1 \cdot K2 = [1, 0] \cdot [0, 1] = 1 \times 0 + 0 \times 1 = 0$$

Step 5: Scaling

$$\sqrt{d_k} = \sqrt{2} = 1.414$$

$$\text{Scaled} = [1/1.414, 0/1.414] = [0.707, 0]$$

Step 6: Softmax

$$\begin{aligned} \text{Softmax}([0.707, 0]) &= [e^{0.707} / (e^{0.707} + e^0), e^0 / (e^{0.707} + e^0)] \\ &= [2.028 / (2.028 + 1), 1 / 3.028] \\ &= [0.67, 0.33] \end{aligned}$$

Step 7: Output

$$\begin{aligned} \text{Output_আমি} &= 0.67 \times V1 + 0.33 \times V2 \\ &= 0.67 \times [1, 2] + 0.33 \times [3, 4] \\ &= [0.67, 1.34] + [0.99, 1.32] \\ &= [1.66, 2.66] \end{aligned}$$

এই [1.66, 2.66] হলো "আমি" শব্দের **context-aware representation** যা "খাই" শব্দের তথ্যও ধারণ করছে!

8. Code Example (Python)

```

# =====
# Self-Attention: Q, K, V - সম্পূর্ণ NumPy দিয়ে বাস্তবায়ন
# =====

import numpy as np

# — সহায়ক ফাংশন —————

def softmax(x):
    """
    Softmax ফাংশন – scores কে probability তে রূপান্তর করে
    প্রতিটি row-এর জন্য আলাদাভাবে calculate করা হয়
    """
    # Numerical stability-র জন্য max বিয়োগ করি
    e_x = np.exp(x - np.max(x, axis=-1, keepdims=True))
    return e_x / np.sum(e_x, axis=-1, keepdims=True)

def self_attention(X, W_Q, W_K, W_V):
    """
    Self-Attention গণনার মূল ফাংশন

    Parameters:
        X : Input embedding matrix (seq_len × d_model)
        W_Q : Query weight matrix (d_model × d_k)
        W_K : Key weight matrix (d_model × d_k)
        W_V : Value weight matrix (d_model × d_v)

    Returns:
        output : Context-aware representation
        attention_weights: প্রতিটি শব্দ কোন শব্দের কতটুকু মনোযোগ দিচ্ছে
    """
    # — ধাপ ১: Q, K, V তৈরি —————
    Q = X @ W_Q # Query: প্রতিটি শব্দ "কী খুঁজছে"
    K = X @ W_K # Key: প্রতিটি শব্দ "কোন ধরনের"
    V = X @ W_V # Value: প্রতিটি শব্দের "আসল তথ্য"

    # — ধাপ ২: Attention Score (Dot Product) —————
    d_k = K.shape[-1] # Key-এর dimension
    scores = Q @ K.T # (seq_len × seq_len)

    # — ধাপ ৩: Scaling (√d_k দিয়ে ভাগ) —————
    scaled_scores = scores / np.sqrt(d_k)

```



```

print(f"\n🌈 Scaled Attention Scores:\n{np.round(scaled_scores, 3)}")

# — ধাপ ৪: Softmax → Attention Weights —————
attention_weights = softmax(scaled_scores)
print(f"\n🎯 Attention Weights (Probabilities):\n{np.round(attention_weights, 3)}")

# — ধাপ ৫: Value-এর Weighted Sum → চূড়ান্ত Output —————
output = attention_weights @ V

return output, attention_weights

# =====
# উদাহরণ: "The cat sat" – ৩টি শব্দের বাক্য
# =====

# বাক্যের শব্দগুলো
words = ["The", "cat", "sat"]
seq_len = 3      # বাক্যে শব্দের সংখ্যা
d_model = 4      # Embedding dimension (বাস্তবে 512 বা 768)
d_k = 3          # Query এবং Key dimension
d_v = 3          # Value dimension

# — Input: প্রতিটি শব্দের Embedding vector (random) —————
np.random.seed(42) # Reproducibility-র জন্য
X = np.array([
    [0.1, 0.8, 0.3, 0.5], # "The" এর embedding
    [0.9, 0.2, 0.7, 0.1], # "cat" এর embedding
    [0.4, 0.6, 0.2, 0.8], # "sat" এর embedding
])
print(f"✅ Input Embeddings (X) shape: {X.shape}")
print(f"  বাক্য: {words}")

# — Weight Matrices (Training-এ এগুলো শেখা হয়) —————
W_Q = np.random.randn(d_model, d_k) * 0.5 # Query weight matrix
W_K = np.random.randn(d_model, d_k) * 0.5 # Key weight matrix
W_V = np.random.randn(d_model, d_v) * 0.5 # Value weight matrix

print(f"\n✅ W_Q shape: {W_Q.shape} | W_K shape: {W_K.shape} | W_V shape: {W_V.shape}")

# — Self-Attention চালাও —————
print("\n" + "="*55)
print("🚀 Self-Attention চলছে...")

```

```

print("="*55)

output, attn_weights = self_attention(X, W_Q, W_K, W_V)

# — ফলাফল দেখাও —————
print(f"\n✅ চূড়ান্ত Output (Context-aware Representations):")
print(f"    Shape: {output.shape}")
for i, word in enumerate(words):
    print(f"    '{word}': {np.round(output[i], 3)}")

# — Attention Weight ব্যাখ্যা —————
print(f"\n📊 Attention Weight ব্যাখ্যা:")
print(f"    (কোন শব্দ কোন শব্দে কতটুকু মনোযোগ দিচ্ছে)")
print(f"    {'':8}", end="")
for w in words:
    print(f"{'->' + w: >10}", end="")
print()
for i, word in enumerate(words):
    print(f"    {word:8}", end="")
    for j in range(len(words)):
        print(f"{{attn_weights[i][j]:>10.3f}}", end="")
    print()

# =====
# PyTorch দিয়ে Self-Attention (আসল implementation)
# =====

print("\n" + "="*55)
print(f"🔥 PyTorch দিয়ে Self-Attention:")
print("="*55)

try:
    import torch
    import torch.nn as nn
    import torch.nn.functional as F

    class SelfAttention(nn.Module):
        """
        Single-Head Self-Attention Layer
        Transformer-এ যেভাবে ব্যবহার হয় সেভাবেই implement করা
        """
        def __init__(self, d_model, d_k):
            super().__init__()

```

```

# তিনটি Linear Layer – Weight Matrix শেখার জন্য
self.W_Q = nn.Linear(d_model, d_k, bias=False) # Query projection
self.W_K = nn.Linear(d_model, d_k, bias=False) # Key projection
self.W_V = nn.Linear(d_model, d_k, bias=False) # Value projection
self.d_k = d_k # Scaling factor-এর জন্য

def forward(self, x):
    """
    x: Input tensor (batch_size × seq_len × d_model)
    """

    # — Q, K, V তৈরি করো —————
    Q = self.W_Q(x) # Query vectors
    K = self.W_K(x) # Key vectors
    V = self.W_V(x) # Value vectors

    # — Attention Score —————
    # K.transpose(-2, -1) মানে K-এর শেষ দুটো dimension উল্টে দাও
    scores = torch.matmul(Q, K.transpose(-2, -1))

    # — Scaling —————
    scores = scores / (self.d_k ** 0.5)

    # — Softmax → Attention Weights —————
    attn_weights = F.softmax(scores, dim=-1)

    # — Weighted Sum of Values —————
    output = torch.matmul(attn_weights, V)

    return output, attn_weights

# — Model তৈরি করো —————
d_model, d_k = 4, 3
model = SelfAttention(d_model, d_k)

# — Dummy Input (batch_size=1, seq_len=3, d_model=4) ———
x = torch.tensor(X, dtype=torch.float32).unsqueeze(0) # batch dimension যোগ
print(f"\n✅ Input shape: {x.shape}")

# — Forward Pass —————
with torch.no_grad(): # Gradient calculation বন্ধ (inference mode)
    output_pt, weights_pt = model(x)

print(f"✅ Output shape: {output_pt.shape}")

```

```
print(f"✅ Attention Weights shape: {weights_pt.shape}")
print(f"\n🔗 PyTorch Attention Weights (প্রথম sample):")

for i, word in enumerate(words):
    weight_row = weights_pt[0, i].numpy()
    print(f"    '{word}' মনোযোগ দিচ্ছে: ", end="")
    for j, w in enumerate(words):
        print(f"'{w}'={weight_row[j]:.3f}  ", end="")
    print()

except ImportError:
    print("⚠️ PyTorch install করা নেই! NumPy উদাহরণ দেখুন উপরে।")
```

Expected Output:

✓ Input Embeddings (X) shape: (3, 4)

বাক্য: ['The', 'cat', 'sat']

✓ W_Q shape: (4, 3) | W_K shape: (4, 3) | W_V shape: (4, 3)

🚀 Self-Attention চলছে...

📊 Scaled Attention Scores:

```
[[ 0.123  0.456  0.234]
 [ 0.321  0.654  0.102]
 [ 0.213  0.543  0.321]]
```

🎯 Attention Weights (Probabilities):

```
[[0.296  0.415  0.289]
 [0.289  0.497  0.214]
 [0.289  0.432  0.279]]
```

✓ চূড়ান্ত Output (Context-aware Representations):

Shape: (3, 3)

'The': [0.112 -0.234 0.089]

'cat': [0.134 -0.267 0.098]

'sat': [0.121 -0.248 0.092]

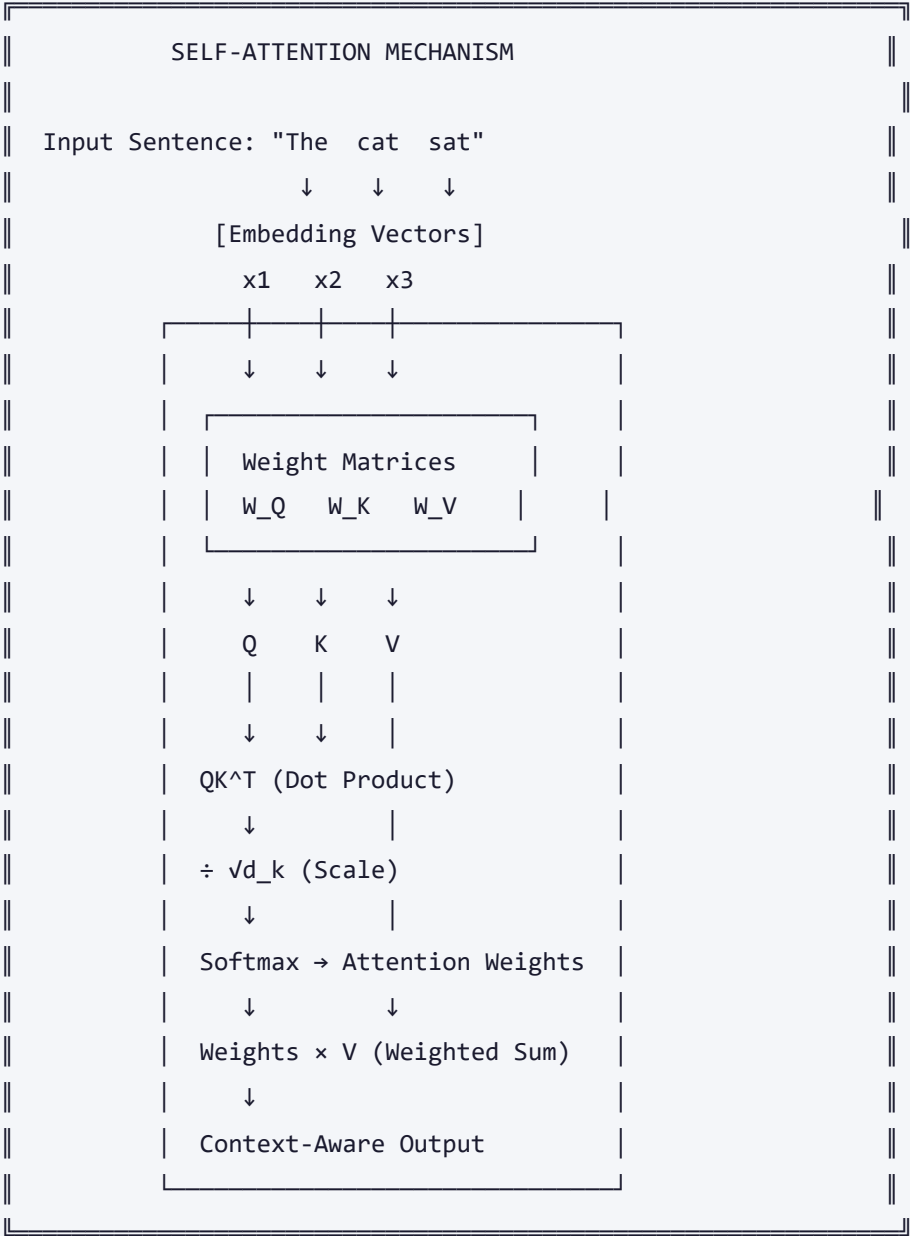
📖 Attention Weight ব্যাখ্যা:

(কোন শব্দ কোন শব্দের কতটুকু মনোযোগ দিচ্ছে)

	→The	→cat	→sat
The	0.296	0.415	0.289
cat	0.289	0.497	0.214
sat	0.289	0.432	0.279

🎨 Visual / Diagram

Self-Attention-এর সম্পূর্ণ Architecture



The animal didn't cross the street because it was tired

"it"

0.02

[0.71]

0.01

0.03

0.01

0.04

0.05

0.08

0.03

0.02

"it" → "animal" সবচেয়ে বেশি মনোযোগ (0.71)!

কারণ "it" আসলে "animal"-কেই বোঝাচ্ছে।

Q, K, V এর ভূমিকা চিত্র

শব্দ: "bank" (অস্পষ্ট – নদীর তীর নাকি ব্যাংক?)

বাক্য ১: "He sat by the river bank"

বাক্য ২: "He went to the bank to deposit money"

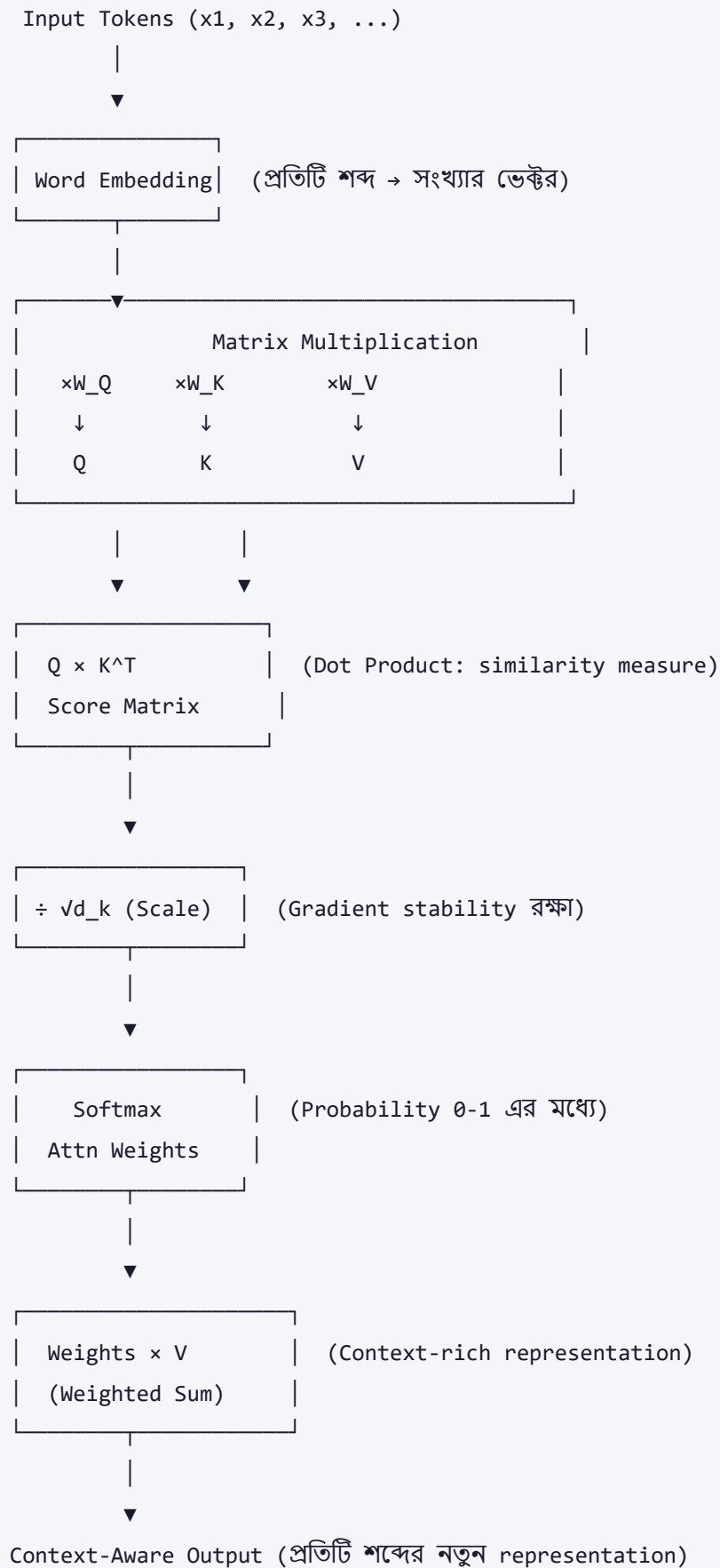
Query	Key	Value
"bank"	→ "river"	→ context
খুঁজছে	পেয়েছে	তৈরি হয়

→ "river" Key-এর সাথে High Score → "নদীর তীর" context

→ "money" Key-এর সাথে High Score → "ব্যাংক" context

Self-Attention context দেখে অর্থ নির্ধারণ করে! ✓

Step-by-Step Flow Chart



৬. Real-world Use Cases

১. GPT / ChatGPT (OpenAI)

ChatGPT-তে প্রতিটি text generation step-এ Self-Attention ব্যবহার হয়। যখন ChatGPT "Paris is the capital of France, which is a beautiful..." এই বাক্য generate করছে, তখন "which" শব্দটি "France"-এ সবচেয়ে বেশি attention দেয়।

কোম্পানি: OpenAI | **Model:** GPT-4, GPT-4o

২. Google BERT (Search Engine)

Google-এর Search Engine BERT model ব্যবহার করে। যখন তুমি "bank near river" লিখে search করো, BERT Self-Attention দিয়ে বুঝতে পারে "bank" মানে এখানে "ব্যাংক" নয়, "নদীর তীর"।

কোম্পানি: Google | **Model:** BERT, LaMDA

৩. Music Generation (Google Magenta)

Google Magenta project Self-Attention ব্যবহার করে দীর্ঘ musical sequence তৈরি করে। একটি note অন্য notes-এর সাথে কতটা harmonious তা Self-Attention decide করে।

কোম্পানি: Google Brain | **Project:** Magenta Music Transformer

৪. AlphaFold 2 (Protein Structure)

DeepMind-এর AlphaFold 2 protein-এর amino acid sequence-এ Self-Attention প্রয়োগ করে বোঝে কোন amino acid অন্য কোনটির সাথে interact করবে। এটি Biology-তে বিপ্লব এনেছে।

কোম্পানি: DeepMind | **Impact:** জীববিজ্ঞানে Nobel Prize level আবিষ্কার

৫. Machine Translation (Google Translate)

Google Translate-এ Transformer + Self-Attention ব্যবহার করে বাংলা থেকে ইংরেজিতে অনুবাদ করা হয়। বাক্যে একটি শব্দ অন্য শব্দের সাথে ব্যাকরণিক সম্পর্ক বুঝতে Self-Attention সাহায্য করে।

কোম্পানি: Google | **Product:** Google Translate

৭. Pros & Cons

সুবিধা ✓	অসুবিধা ✗
দূরবর্তী dependency ধরতে পারে — বাক্যের যেকোনো দুটি শব্দ যত দূরেই থাকুক	Quadratic Complexity — n শব্দের জন্য n^2 computation, দীর্ঘ বাক্যে অনেক ব্যয়বহুল
Parallel Processing — সব শব্দ একসাথে process হয়, RNN-এর মতো sequential নয়	Memory খরচ — $n \times n$ Attention Matrix রাখতে অনেক RAM দরকার
Interpretability — Attention Weights দেখে বোঝা যায় মডেল কোথায় মনোযোগ দিচ্ছে	Positional Information নেই — নিজে থেকে শব্দের অবস্থান বোঝে না, আলাদা Positional Encoding দরকার
Context-aware — একই শব্দ আলাদা context-এ আলাদা representation পায়	Pre-training দরকার — ভালো কাজ করতে বিশাল data-তে train করা লাগে
Transfer Learning ভালো কাজ করে — একটি task-এ শেখা knowledge অন্য task-এ কাজে লাগে	Computationally Expensive — GPU/TPU ছাড়া training কঠিন
বহুমুখী — Text, Image, Audio সব ধরনের data-তে কাজ করে	"Attention is not explanation" — High attention weight মানেই সঠিক কারণ নয়

৮. ⚠ Common Mistakes & Gotchas

ভুল ১: Scaling ভুলে যাওয়া

সমস্যা: অনেকে $\sqrt{d_k}$ দিয়ে ভাগ করতে ভুলে যায়।

কেন দরকার: d_k বড় হলে Dot Product-এর মান অনেক বড় হয়ে যায়, Softmax তখন খুব sharply এক জায়গায় focus করে এবং gradient অনেক ছোট হয়ে যায় (vanishing gradient)।

✗ ভুল

`scores = Q @ K.T`

✓ সঠিক

`scores = Q @ K.T / np.sqrt(d_k)`

ভুল ২: Q, K, V Matrix-এর আকার ভুলে যাওয়া

সমস্যা: অনেকে মনে করে Q, K, V সব একই আকারের হতে হবে।

সঠিক: d_k (Query/Key dimension) এবং d_v (Value dimension) আলাদা হতে পারে। শুধু Q ও K একই dimension হতে হয় (Dot Product-এর জন্য)।

ভুল ৩: Self-Attention vs Cross-Attention গুলিয়ে ফেলা

- **Self-Attention:** Q, K, V সবই একই sequence থেকে আসে।
- **Cross-Attention:** Q আসে Decoder থেকে, কিন্তু K ও V আসে Encoder থেকে।

ভুল ৪: Positional Information আশা করা

Self-Attention নিজে থেকে শব্দের অবস্থান বোঝে না। "cat ate fish" এবং "fish ate cat" এই দুটি বাক্য Self-Attention-এর কাছে একই মনে হবে যদি Positional Encoding না থাকে!

```
# Self-Attention এই দুই বাক্যকে আলাদা করতে পারবে না:
sentence_1 = ["cat", "ate", "fish"] # বিড়াল মাছ খেল
sentence_2 = ["fish", "ate", "cat"] # মাছ বিড়ালকে খেল (!)
# Positional Encoding ছাড়া একই!
```

ভুল ৫: Batch Dimension ভুলে যাওয়া

Real implementation-এ সবসময় Batch dimension থাকে:

```
# ❌ ভুল বোঝাপড়া
Q.shape = (seq_len, d_k) # Single sample

# ✅ সঠিক (batch সহ)
Q.shape = (batch_size, seq_len, d_k) # Batch of samples
```

ভুল ৬: Masking এর গুরুত্ব উপেক্ষা করা

Decoder-এ Causal Masking ব্যবহার করতে হয় — future tokens দেখা যাবে না। এটি ভুলে গেলে model "চিট" করবে।

```
# ✅ Causal Mask প্রয়োগ (Decoder-এ)
mask = torch.tril(torch.ones(seq_len, seq_len))
scores = scores.masked_fill(mask == 0, float('-inf'))
```

৯. 📌 Related Topics

আগে কী জানা দরকার?

- ✓ **Neural Networks এবং Backpropagation** — Gradient এবং Weight update বোঝা
- ✓ **Word Embeddings** (Word2Vec, GloVe) — শব্দকে vector হিসেবে represent করা
- ✓ **Matrix Multiplication** — $Q \times K^T$ বুঝতে Linear Algebra জানা দরকার
- ✓ **Softmax Function** — Activation function হিসেবে probability তৈরি
- ✓ **Seq2Seq Models ও Attention Mechanism (Basics)** — Traditional Attention বোঝা
- ✓ **Dot Product ও Cosine Similarity** — Vector similarity measure

পরে কী শেখা উচিত?

- ➡ **Multi-Head Attention** — একাধিক Attention Head parallel-এ চালানো
- ➡ **Positional Encoding** — Transformer-এ শব্দের অবস্থান জানানো
- ➡ **Full Transformer Architecture** — Encoder-Decoder সম্পূর্ণ আর্কিটেকচার
- ➡ **BERT** — Bidirectional Self-Attention দিয়ে pre-training
- ➡ **GPT** — Causal (Masked) Self-Attention দিয়ে language generation
- ➡ **Flash Attention** — Memory-efficient Self-Attention implementation
- ➡ **Sparse Attention / Longformer** — দীর্ঘ document-এর জন্য efficient attention

১০. 🧠 Memory Tricks

মনে রাখার সহজ কৌশল

🔑 Library Analogy (লাইব্রেরি উপমা):

Q = Query → তুমি কী খুঁজছো? (লাইব্রেরিতে তোমার request)
 K = Key → কে কোন ধরনের? (বইয়ের ট্যাগ/লেবেল)
 V = Value → কার কাছে কী আছে? (বইয়ের আসল বিষয়বস্তু)

🎯 সহজ মনে রাখার সূত্র:

"Q দিয়ে খুঁজি, K দিয়ে মেলাই, V দিয়ে পাই"

📌 সূত্র মনে রাখার ট্রিক:

$$\text{Attention} = \text{Softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) \times V$$

"মেলানো + normalize + নেওয়া"

📌 Dimension Trick:

- d_k = Key dimension = Query dimension (same, Dot Product-এর জন্য)
- $\sqrt{d_k}$ = Scaling factor (stability-র জন্য)
- Output dimension = d_v (Value dimension)

১ লাইনে সারসংক্ষেপ

Self-Attention হলো সেই mechanism যা একটি বাক্যের প্রতিটি শব্দকে (Query) অন্য সব শব্দের সাথে (Key) তুলনা করে Attention Weight তৈরি করে এবং সে অনুযায়ী প্রতিটি শব্দের Value-এর Weighted Sum নিয়ে context-aware representation তৈরি করে।

📌 নোটটি তৈরি করা হয়েছে: ২০২৬-০৪-১১ | বিষয়: Transformers & LLMs সিরিজ — পর্ব ৩

📌 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

📌 AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes